ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №9

«Паралельність, багатопроцесність та оптимізація»

з курсу «Професійний Python»
Варіант №13 — «Система обліку комп'ютерної техніки»

Виконав: Капарник Дмитро Андрійович

Група: ФЕП-32

Перевірив: доцент Куньо Іван Михайлович

Дата: 18.10.2026

Львів — 2026

1. Мета роботи

Виміряти вузькі місця й порівняти послідовні, потокові, багатопроцесні та векторизовані реалізації з перевіркою еквівалентності.

2. Індивідуальний варіант і постановка задачі

Варіант №13 — система обліку комп'ютерної техніки. Профілювати пошук, фільтрацію, підсумки та групування обладнання; дослідити ThreadPoolExecutor, ProcessPoolExecutor, NumPy й кеш з інвалідацією. Виконати п'ять повторів на трьох розмірах та кількох workers.

Наскрізний проєкт продовжує ЛР №1. Базові поля: інвентарний номер, тип, виробник, модель і вартість. Збережено пошук, фільтр виробника, середню/загальну вартість і визначення найдорожчого пристрою.

3. Структура проєкту та створені модулі

```
inventory-service/
  pyproject.toml, README.md, .gitignore
  src/computer_inventory/
    performance.py
    analytics.py
    services.py
    streaming.py
    database.py
  src/inventory_service/  # фінальний пакет запуску
  tests/
  scripts/
  docs/GITHUB.md
```

4. Профілювання предметного сценарію

Сценарій на 100 000 Device включає пошук, фільтр Dell, групування за типом і повну статистику. cProfile зафіксував 1 700 060 викликів за 0.574 с. Найбільше накопиченого часу має search_devices: 0.333 с, приблизно 58% сценарію. Її власний tottime — 0.054 с; решта припадає на any, generator expression та casefold. Пошук підрядка в кількох полях залишається O(n).

```
         1700060 function calls (1700042 primitive calls) in 0.574 seconds

   Ordered by: cumulative time
   List reduced from 37 to 20 due to restriction <20>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.002    0.002    0.574    0.574 scripts\lab09.py:67(workload)
        1    0.054    0.054    0.333    0.333
src\computer_inventory\services.py:19(search_devices)
   100000    0.083    0.000    0.279    0.000 {built-in method builtins.any}
   499997    0.130    0.000    0.196    0.000
src\computer_inventory\services.py:24(<genexpr>)
```

```
        1    0.001    0.001    0.126    0.126
src\computer_inventory\analytics.py:19(wrapped)
        1    0.001    0.001    0.125    0.125
src\computer_inventory\analytics.py:74(summarize)
   499999    0.082    0.000    0.082    0.000 {method 'casefold' of 'str' objects}
        1    0.048    0.048    0.075    0.075
src\computer_inventory\analytics.py:61(group_by)
        2    0.000    0.000    0.065    0.032
src\computer_inventory\services.py:37(total_cost)
        2    0.042    0.021    0.065    0.032 {built-in method builtins.sum}
```

5. Оптимізація точного пошуку номера

Метод	Записів	Запитів	Час, с
linear	100000	200	0.973384
indexed	100000	200	0.000127

Побудова індексу зайняла 0.018169 с. Прискорення повторних пошуків: 7670.5×; з урахуванням одноразової побудови — 53.2×. Обидва способи шукають повний інвентарний номер й повертають ті самі Device. Це окремий контракт від пошуку підрядка в усіх полях. У фінальному API точний inventory_number використовує UNIQUE-індекс SQLite; загальний консольний пошук зберігає потрібну поведінку підрядка.

```
def build_index(devices: Iterable[Device]) -> dict[str, Device]:
    items = list(devices)
    index = {d.inventory_number.casefold(): d for d in items}
    if len(items) != len(index):
        raise ValueError("Duplicate inventory number")
    return index
```

6. Порівнювані числові реалізації

Для числового підзавдання формуються 10 000, 100 000 і 1 000 000 цілих цін у копійках: 1000 + i mod 90000. Контракт — (кількість, сума, сума квадратів, максимум). Контроль меж NumPy запобігає переповненню int64. Час NumPy включає перетворення list в array та перевірку діапазону; час процесів — запуск пулу, серіалізацію й збирання результатів. Кожен запуск перевіряє точний збіг цілих підсумків.

```
def numeric_statistics(values: list[int]) -> tuple[int, int, int, int]:
    if not values:
        return 0, 0, 0, 0
    return len(values), sum(values), sum(v * v for v in values), max(values)

def combine(parts: Iterable[tuple[int, int, int, int]]) -> tuple[int, int, int, int]:
    rows = list(parts)
    return (
        sum(p[0] for p in rows),
        sum(p[1] for p in rows),
        sum(p[2] for p in rows),
```

```
        max((p[3] for p in rows), default=0),
    )

def parallel_statistics(
    values: list[int], workers: int, processes: bool = False
) -> tuple[int, int, int, int]:
    if workers < 1:
        raise ValueError("Workers must be positive")
    size = max(1, (len(values) + workers - 1) // workers)
    chunks = [values[i : i + size] for i in range(0, len(values), size)]
    executor = ProcessPoolExecutor if processes else ThreadPoolExecutor
    with executor(max_workers=workers) as pool:
        return combine(pool.map(numeric_statistics, chunks))

def numpy_statistics(values: list[int]) -> tuple[int, int, int, int]:
    if not values:
        return 0, 0, 0, 0
    # Check bounds before using int64 so vectorization cannot silently overflow.
    bound = max(abs(min(values)), abs(max(values)))
    if bound * bound * len(values) > np.iinfo(np.int64).max:
        raise ValueError("Input exceeds safe int64 range")
    array = np.asarray(values, dtype=np.int64)
    return (
        len(values),
        int(array.sum()),
        int(np.square(array).sum()),
        int(array.max(axis=0)),
    )
```

7. Результати п’яти повторів та speedup

Набір 10 000 записів

Метод	W	1, с	2, с	3, с	4, с	5, с	Середнє, с	S
Sequential	1	0.00052	0.00051	0.00051	0.00052	0.00051	0.00051	1.000
NumPy	1	0.00051	0.00041	0.00046	0.00061	0.00038	0.00047	1.087
Threads	1	0.00205	0.00097	0.00109	0.00085	0.00093	0.00118	0.436
Processes	1	0.52377	0.44225	0.46804	0.46841	0.43177	0.46685	0.001
Threads	2	0.00143	0.00096	0.00101	0.00121	0.00119	0.00116	0.442
Processes	2	0.49264	0.46824	0.48790	0.45251	0.47131	0.47452	0.001
Threads	4	0.00166	0.00135	0.00148	0.00142	0.00125	0.00143	0.358
Processes	4	0.54694	0.60404	0.54930	0.59590	0.56471	0.57218	0.001
Threads	8	0.00200	0.00172	0.00163	0.00151	0.00163	0.00170	0.302
Processes	8	0.91905	0.90917	0.82151	0.80844	0.78731	0.84910	0.001

Набір 100 000 записів

Метод	W	1, с	2, с	3, с	4, с	5, с	Середнє, с	S
Sequential	1	0.01102	0.01133	0.01080	0.01083	0.00723	0.01024	1.000

Метод	W	1, с	2, с	3, с	4, с	5, с	Середнє, с	S
NumPy	1	0.00537	0.00576	0.00593	0.00622	0.00654	0.00597	1.717
Threads	1	0.01174	0.01186	0.01256	0.00729	0.01221	0.01113	0.920
Processes	1	0.43173	0.44535	0.41117	0.44872	0.46705	0.44080	0.023
Threads	2	0.01075	0.00980	0.01168	0.01143	0.01186	0.01111	0.922
Processes	2	0.49565	0.47889	0.48021	0.47250	0.48587	0.48262	0.021
Threads	4	0.01214	0.01213	0.01188	0.01157	0.01113	0.01177	0.870
Processes	4	0.54052	0.58177	0.56766	0.60680	0.55705	0.57076	0.018
Threads	8	0.01236	0.01196	0.01231	0.01273	0.01146	0.01216	0.842
Processes	8	0.83077	0.83041	0.84724	0.83663	0.79956	0.82892	0.012

Набір 1 000 000 записів

Метод	W	1, с	2, с	3, с	4, с	5, с	Середнє, с	S
Sequential	1	0.12716	0.12552	0.11729	0.11916	0.11706	0.12124	1.000
NumPy	1	0.06607	0.06611	0.06640	0.06494	0.06849	0.06640	1.826
Threads	1	0.13738	0.12450	0.12388	0.13130	0.12469	0.12835	0.945
Processes	1	0.55229	0.56447	0.60182	0.59489	0.57464	0.57762	0.210
Threads	2	0.12406	0.13116	0.13175	0.12593	0.11679	0.12594	0.963
Processes	2	0.58768	0.55322	0.57732	0.56552	0.56778	0.57030	0.213
Threads	4	0.10409	0.11578	0.12617	0.12994	0.12392	0.11998	1.010
Processes	4	0.65013	0.64214	0.58816	0.62714	0.67034	0.63558	0.191
Threads	8	0.13043	0.12837	0.12368	0.11775	0.11939	0.12392	0.978
Processes	8	0.80212	0.90363	0.89151	0.85403	0.85744	0.86175	0.141

S = T_sequential / T_method. Значення менше 1 означає уповільнення. Python 3.12.14, Windows, os.cpu_count() = 12; одночасне навантаження системи не контролювалося. Короткі операції особливо чутливі до шуму, тому близькі до 1 значення не доводять стабільного прискорення.

8. Пам’ять, threading, імпорт та кеш

Записів	Python peak, байт	NumPy peak, байт
10000	496	160380
100000	572	1600380
1000000	572	16000380

Пам’ять виміряно після створення початкового list, тому таблиця показує додаткові алокації числового виклику, а не весь набір даних. NumPy створює масив і проміжний результат. tracemalloc не охоплює повний RSS та пам’ять усіх дочірніх процесів, отже ці значення не є порівнянням загальної пам’яті пулів.

```
def import_files(paths: list[Path], workers: int = 4) -> int:
    def count_file(path: Path) -> int:
        return aggregate(pipeline(path)).count
```

```
    with ThreadPoolExecutor(max_workers=workers) as pool:
        return sum(pool.map(count_file, paths))

def synchronized_count(workers: int = 4, increments: int = 1000) -> int:
    lock = Lock()
    total = 0

    def worker() -> None:
        nonlocal total
        for _ in range(increments):
            with lock:
                total += 1

    threads = [Thread(target=worker) for _ in range(workers)]
    for thread in threads:
        thread.start()
    for thread in threads:
        thread.join()
    return total

class CachedInventory:
    def __init__(self) -> None:
        self._items: dict[str, Device] = {}
        self._cached = lru_cache(maxsize=128)(self._calculate)

    def _calculate(self, manufacturer: str, device_type: str) -> dict[str, Any]:
        return summarize(
            d
            for d in self._items.values()
            if (not manufacturer or d.manufacturer.casefold() == manufacturer)
            and (not device_type or d.device_type.casefold() == device_type)
        )

    def add(self, device: Device) -> None:
        key = device.inventory_number.casefold()
        if key in self._items:
            raise ValueError("Duplicate inventory number")
        self._items[key] = device
        self._cached.cache_clear()

    def delete(self, inventory_number: str) -> None:
        del self._items[inventory_number.casefold()]
        self._cached.cache_clear()

    def change_cost(self, inventory_number: str, cost: Decimal) -> None:
        key = inventory_number.casefold()
        self._items[key] = replace(self._items[key], cost=cost)
        self._cached.cache_clear()

    def statistics(
        self, manufacturer: str = "", device_type: str = ""
    ) -> dict[str, Any]:
        result = self._cached(
            manufacturer.strip().casefold(), device_type.strip().casefold()
        )
```

```
        # Do not let callers mutate the cached dictionary or nested values.
        import copy

        return copy.deepcopy(result)

    def cache_info(self) -> Any:
        return self._cached.cache_info()
```

Паралельний імпорт чотирьох inventory-файлів по 10 000 записів повернув 40000 пристроїв за 0.3656 с. Приклад Thread із Lock дав рівно 4000 приростів. Cold-cache статистика виробника: 0.041517 с; warm-cache: 0.00001940 с, прискорення одного повторного виклику 2140.1×. Це коротке вимірювання, його точність обмежена роздільністю таймера й системним шумом.

Ключ кешу містить виробника й тип; add, delete і change_cost очищають кеш. Тест підтвердив зміну підсумку після зміни ціни, видалення й додавання; результат повертається як копія, тому клієнт не може пошкодити кеш. cache_info у журналі наприкінці сценарію нульовий, бо остання мутація вже очистила накопичені hit/miss.

9. Аналіз та перевірка правильності

Для 10000 записів найменше середнє серед виміряних конфігурацій: NumPy (1 workers), 0.000472 с. NumPy має speedup 1.087. Пули процесів у цьому підзавданні не компенсували витрати запуску й передавання даних.

Для 100000 записів найменше середнє серед виміряних конфігурацій: NumPy (1 workers), 0.005966 с. NumPy має speedup 1.717. Пули процесів у цьому підзавданні не компенсували витрати запуску й передавання даних.

Для 1000000 записів найменше середнє серед виміряних конфігурацій: NumPy (1 workers), 0.066402 с. NumPy має speedup 1.826. Пули процесів у цьому підзавданні не компенсували витрати запуску й передавання даних.

CPU-цикл у звичайному CPython обмежений GIL для потоків; multiprocessing обходить цю межу, але потребує крупніших задач. Для production доречні SQL-агрегати й індексований точний пошук, а NumPy — для великих пакетних числових розрахунків після вимірювання. Кеш застосовується лише з контрольованою інвалідацією. ThreadPool залишено для незалежного файлового імпорту.

```
@pytest.mark.parametrize("values", [[], [1], [1, 2, 3, 4], [100, 200, 500]])
def test_optimized_equivalence(values):
    expected = numeric_statistics(values)
    assert numpy_statistics(values) == expected
    assert parallel_statistics(values, 2) == expected

def test_process_equivalence():
    assert parallel_statistics(list(range(100)), 2, True) == numeric_statistics(
        list(range(100))
```

```
    )

def test_cache_invalidation(devices):
    cache = CachedInventory()
    for device in devices:
        cache.add(device)
    first = cache.statistics("Dell")
    assert cache.statistics("Dell") == first
    assert cache.cache_info().hits == 1
    cache.change_cost("INV-002", Decimal(1))
    assert cache.statistics("Dell")["total"] != first["total"]
    cache.delete("INV-002")
    assert cache.statistics("Dell")["count"] == 1
    cache.add(devices[1])
    assert cache.statistics("Dell")["count"] == 2
    with pytest.raises(ValueError):
        cache.add(devices[1])
    result = cache.statistics("Dell")
    result["count"] = 999
    assert cache.statistics("Dell")["count"] == 2
```

Код і історія лабораторних: https://github.com/dimakaparnyk/professional-python-labs. Для відповідного етапу використовується тег lab-09, фінальна інтегрована реалізація — main (перевірений CI commit 2e3fb3d).

10. Відповіді на контрольні запитання

1. Що таке concurrency?

Організація кількох задач так, що їхнє виконання перекривається в часі.

2. Що таке parallelism?

Фактичне одночасне виконання обчислень кількома виконавцями.

3. Чим concurrency відрізняється від parallelism?

Concurrency може чергувати задачі на одному ядрі; parallelism потребує одночасного виконання.

4. Що таке process?

Окремий запущений екземпляр програми з власним адресним простором.

5. Що таке thread?

Потік виконання всередині процесу, який поділяє його пам’ять.

6. Чим process відрізняється від thread?

Процеси ізольовані та потребують обміну даними; потоки одного процесу мають спільну пам’ять.

7. Що таке GIL?

Global Interpreter Lock — механізм блокування інтерпретатора в традиційній збірці CPython.

8. Яку роль GIL відіграє у CPython?

Обмежує одночасне виконання Python-байткоду потоками одного інтерпретатора; існують також free-threaded збірки.

9. Що таке CPU-bound задача?

Задача, тривалість якої переважно визначає швидкість обчислень процесора.

10. Наведіть приклад CPU-bound задачі.

Підрахунок числової статистики великим циклом Python.

11. Що таке I/O-bound задача?

Задача, у якій основний час витрачається на очікування введення чи виведення.

12. Наведіть приклад I/O-bound задачі.

Завантаження даних мережею або паралельне читання незалежних файлів.

13. Коли доцільно застосовувати threading?

Для перекриття I/O-очікування або роботи з бібліотеками, що звільняють GIL.

14. Коли доцільно застосовувати multiprocessing?

Для достатньо великих незалежних CPU-bound задач, коли виграш перевищує витрати процесів.

15. Для чого використовується Thread?

Для створення й керування окремим потоком.

16. Для чого використовується start()?

Починає виконання функції в новому потоці або процесі відповідного об’єкта.

17. Для чого використовується join()?

Очікує завершення потоку або процесу.

18. Що таке race condition?

Залежність результату від неконтрольованого порядку доступу до спільного стану.

19. Що таке critical section?

Ділянка роботи зі спільним ресурсом, яка потребує узгодженого доступу.

20. Для чого використовується Lock?

Для взаємного виключення доступу до критичної ділянки.

21. Що таке synchronization?

Узгодження виконання задач і доступу до спільних даних.

22. Чи завжди synchronization покращує performance?

Ні; блокування додає витрати, але потрібне для коректності за спільного змінюваного стану.

23. Що таке ThreadPoolExecutor?

Керований пул потоків з інтерфейсом Future, submit та map.

24. Коли його доцільно використовувати?

Для незалежних I/O-операцій і задач із прийнятними витратами синхронізації.

25. Що таке ProcessPoolExecutor?

Керований пул окремих процесів для виконання незалежних задач.

26. Коли його доцільно використовувати?

Для великих CPU-bound задач, які можна серіалізувати та розподілити.

27. Для чого використовується executor.map()?

Для застосування функції до послідовності аргументів із поверненням результатів у вхідному порядку.

28. Що таке worker?

Окремий потік або процес, який обробляє задачі пулу.

29. Чи завжди більша кількість workers дає більшу швидкість?

Ні; зайві виконавці додають перемикання, блокування, витрати пам’яті та конкуренцію за ресурси.

30. Що таке multiprocessing overhead?

Витрати запуску, IPC, серіалізації, копіювання даних і збирання результатів.

31. Що таке serialization?

Перетворення даних у формат передавання або збереження, наприклад pickle для процесів.

32. Чому multiprocessing може бути повільним для маленьких задач?

Запуск і передавання даних можуть тривати довше за саме обчислення.

33. Для чого використовується timeit?

Для повторних вимірювань коротких фрагментів коду з меншим стороннім шумом.

34. Для чого використовується perf_counter()?

Для вимірювання проміжків часу високоточним монотонним таймером.

35. Навіщо виконувати декілька benchmark repeats?

Щоб оцінити коливання й не робити висновок з випадкового одного запуску.

36. Що таке profiling?

Вимірювання розподілу витрат між функціями та ділянками програми.

37. Для чого використовується cProfile?

Для детермінованого профілювання викликів і часу Python-функцій.

38. Що показує ncalls?

Кількість викликів функції; окремо можуть показуватися примітивні й рекурсивні виклики.

39. Що показує tottime?

Власний час функції без часу вкладених викликів.

40. Що показує cumtime?

Накопичений час функції разом із викликаними нею функціями.

41. Що таке bottleneck?

Компонент, який обмежує загальну швидкість виконання сценарію.

42. Як знайти bottleneck?

Профілювати реальний сценарій і знайти функції з найбільшим внеском у час або ресурсні витрати.

43. Для чого використовується tracemalloc?

Для відстеження алокацій пам’яті Python і пікового використання під час експерименту.

44. Що таке peak memory?

Найбільший зафіксований обсяг відстежуваної пам’яті за інтервал; це не весь RSS процесу.

45. Що таке NumPy?

Бібліотека масивів і числових обчислень із низькорівневими реалізаціями операцій.

46. Що таке vectorization?

Виконання операції над масивом через спеціалізований цикл бібліотеки замість циклу Python.

47. Чому NumPy часто швидший за Python loops?

Усуває витрати Python-інтерпретації для кожного елемента й ефективно використовує типізовану пам’ять.

48. Що таке caching?

Збереження раніше обчисленого результату для повторного використання.

49. Для чого використовується @cache?

Для необмеженого за кількістю ключів кешування результатів функції з hashable аргументами.

50. Для чого використовується @lru_cache?

Для кешу з обмеженням розміру та витісненням давно не використаних ключів.

51. Що таке cache hit?

Звернення, для якого результат уже знайдено в кеші.

52. Що таке cache miss?

Звернення, яке вимагає нового обчислення.

53. Що таке cache invalidation?

Видалення або оновлення кешу після зміни даних, від яких залежить результат.

54. Коли кешування може бути небезпечним?

Коли дані змінюються без інвалідації, ключ не враховує залежностей або кеш зростає без контролю.

55. Як обчислюється speedup?

S = час базової реалізації / час порівнюваної реалізації за однакового контракту.

56. Що означає speedup менше 1?

Порівнювана реалізація працює повільніше за базову.

57. Чому threaded CPU-bound version може бути повільнішою?

Через GIL у звичайному CPython та додаткові витрати пулу, планування й синхронізації.

58. Чому ProcessPoolExecutor може прискорити CPU-bound код?

Кожен процес має власний інтерпретатор, тому обчислення можуть виконуватися на різних ядрах.

59. Чому необхідно перевіряти правильність optimized implementation?

Оптимізація може змінити округлення, порядок, типи чи втратити дані; результати повинні збігатися.

60. Чому оптимізацію потрібно починати з profiling?

Щоб прискорювати реальне вузьке місце, а не витрачати час на несуттєву ділянку.

11. Висновок

Визначено внесок пошуку у час предметного сценарію, виміряно індексоване прискорення точного пошуку та порівняно чотири числові реалізації. Повторні експерименти показали, що більша кількість виконавців не гарантує виграшу. Коректність оптимізованих підсумків і кеш-інвалідації підтверджено тестами.